# Project Synapse: Org-Wide Zero-ETL Semantic Data Core

**UNIFIED MASTER ARCHITECTURAL SPECIFICATION & IMPLEMENTATION ROADMAP**

Consolidated master · Original blueprint · Gemini control-plane formalisms · Org-wide completeness design
Status: Architecture thesis / production-completeness specification (no petabyte simulation required)

**Source corpus:** Zero_ETL_Semantic_Data_Blueprint.pdf · Project_Synapse_Master_Architecture.pdf (Gemini expansion) · ORG_WIDE_SEMANTIC_DATA_CORE.md · Prior architectural analysis

## 0. Abstract: Architectural Completeness Goal

Project Synapse demonstrates architectural feasibility for an enterprise-wide, discrepancy-tolerant, schema-on-read context platform. Instead of verifying petabyte-scale compute arrays, this document guarantees **production completeness** by addressing structural vulnerabilities—deterministic parsing, inference token blowouts, dynamic access controls, query latency, entity resolution, conflict honesty, evaluation gates, and multi-domain governance—with concrete software primitives.

**Success for this phase of work:** every production hole is named, owned by a subsystem, and has a design-level mitigation—not a hand-wave. The question is whether org-wide semantic data management is **doable with the right infra and effort**. This specification answers yes—as a phased platform coexisting with warehouses—not as a big-bang ETL replacement.

> **What We Are Proving (Without Full Infra)**
> Data-plane contracts and discrepancy models handle variety; determinism, citations, and eval keep answers trustworthy; ABAC on raw blocks secures multi-tenant access; tiered retrieval and budgets bound cost/latency; versioned ontologies and idempotent reprocess allow evolution without pipeline rewrites.

## 1. The Problem Statement: The Schema Bottleneck

In modern enterprise software engineering, traditional data pipelines act as a major structural bottleneck. Operational workloads generate chaotic, distributed, and highly transactional data (OLTP). To make this data usable for search, analysis, or intelligence, organizations historically enforce a strict **Schema-on-Write** paradigm.

This requires building complex, multi-stage ETL/ELT pipelines using heavyweight compute orchestration layers (e.g., Apache Spark, dbt, Airflow). Data must be pulled out of operational environments, cleaned, normalized, deduplicated, and mapped into predefined relational tables or data warehouse schemas.

### 1.1 Core Inefficiencies of Traditional Pipelines

| Failure Mode | Effect |
|---|---|
| **Brittle Infrastructure** | A single upstream modification to an operational database schema breaks downstream transformations, creating immediate production downtime. |
| **Latency Delays** | Processing batch jobs introduces time lags, separating the moment data is created from the moment it becomes actionable. |
| **Contextual Destruction** | Forced schema normalization strips away semantic anomalies, edge cases, and natural-language nuances that are highly valuable to reasoning engines. |
| **Premature Agreement** | Warehouses force artificial consensus across domains (finance, eng, sales, legal) that inherently view data differently—blocking org-wide integration. |

With foundational Large Language Models featuring massive context windows ($C \geq 10^6$ tokens), runtime attention mechanisms, and real-time semantic synthesis, the necessity of rigid preprocessing is open for debate. The refined hypothesis is not “dump everything into long context,” but:

> **The Synapse Core Thesis**
> Store raw operational telemetry permanently alongside rich lineage tags. Defer structural layout and validation logic to query time via a schema-on-read execution model. Maintain an evolving graph of **entities, facts, and raw conflicts** rather than a single cleaned database as the singular source of absolute truth. Apply cheap indexes, temporal graphs, and selective LLM synthesis under explicit cost and latency budgets.

**Naming discipline:** This is *minimal, reversible prep + schema-on-read*, not literally zero work. “Zero-ETL” means zero forced warehouse schema as the price of admission—not zero transformation. Data-Juicer-class operators remain light prep; the destination is not a rigid warehouse table.

### 1.2 Org-Wide Reality: Variety & Discrepancy

Organization-scale data is not merely large—it is politically and semantically multi-owner:

• **Heterogeneous** — OLTP DBs, SaaS exports, logs, tickets, PDFs, email, metrics, events, spreadsheets, APIs
• **Discrepant** — same entity, different IDs, conflicting attributes, stale copies, partial truth
• **Politically multi-owner** — finance, eng, sales, legal each “own” a slice
• **Mixed consumers** — humans, BI, agents, workflows, compliance audits

| Dimension | Design Target (order-of-magnitude) |
|---|---|
| **Domains** | 10–50 business domains (CRM, billing, infra, HR, support, …) |
| **Source systems** | 50–500 systems / connectors |
| **Object volume** | 10^8–10^11 objects / events over retention window |
| **Query consumers** | Interactive agents, batch synthesis, audit export |
| **Latency classes** | Hot (seconds), warm (tens of seconds), cold (minutes, async) |
| **Consistency** | Eventual for graph; strong lineage to raw; explicit conflicts |
| **Tenancy** | Org + BU + team + user ABAC; regulated partitions |

These are sizing assumptions for architecture, not SLAs we can simulate without infra.

### 1.3 Non-Goals (Honesty Boundaries)

• Not a replacement for OLTP systems of record
• Not the first place for sub-10ms transactional reads
• Not a guarantee of perfect truth without human governance for regulated numbers
• Not “one embedding store for everything”

Synapse sits **above** systems of record as the semantic integration and reasoning plane.

## 2. Design Principles (Org-Wide)

| # | Principle | Meaning |
|---|---|---|
| **1** | Raw is sacred | Immutable landing zone; never mutate source bytes. All intelligence is derived. |
| **2** | Lineage is mandatory | Every claim points to source block(s) + extraction version + time. |
| **3** | Discrepancy is first-class | Conflicts are stored and queryable, not silently “resolved away.” |
| **4** | Schema emerges, then stabilizes | Soft ontology early; governed contracts where consumers demand them. |
| **5** | Tiered intelligence | Cheap local models/rerankers by default; large LLMs selectively. |
| **6** | Budgeted reasoning | Every query has a cost/latency budget and a degraded answer path. |
| **7** | Security follows the block | ACL/ABAC tags travel with raw and all derivatives. |
| **8** | Reprocess is normal | Better extractors re-run over history without dual-write nightmares. |
| **9** | Eval is product | Continuous answer quality gates; no silent regression. |

| # | Principle | Meaning |
|---|---|---|
| **10** | Human correction is a write path | Experts pin, merge, or reject facts; those are higher-trust edges. |

## 3. Open-Source Ecosystem Components (Engine Pieces)

To remove conventional warehouse-bound ETL overhead, Project Synapse orchestrates four specialized open-source core frameworks. They are **necessary but not sufficient** at org scale—the control plane (§6) and enterprise subsystems (§5) complete the platform.

### 1. Graphiti (by Zep)

`github.com/getzep/graphiti`

**Enterprise Stack Role: Continuous Context & Temporal Relation Modeler.**

Utilizes a Learned Ontology model to parse data sequentially ("episodes"), dynamically indexing evolving entity connections and temporal states without manual schema declarations. At org scale: partition by domain/tenant; apply episode backpressure; pair with entity resolution.

*Puzzle piece: defeats lack of temporal cross-reference memory; links isolated dirty inputs into a fluid, evolving graph.*

### 2. GraphRAG (by Microsoft Research)

`github.com/microsoft/graphrag`

**Enterprise Stack Role: Hierarchical Abstractor & Global Query Synthesizer.**

Clusters unstructured text into hierarchical communities and dynamic abstractions, enabling global thematic queries ("What are the top failure modes across all system logs?") where localized vector lookups fail. Prefer offline/async jobs—not default interactive path.

*Puzzle piece: resolves global analysis; cross-domain synthesis without predefined warehouse schemas.*

### 3. Data-Juicer (by Alibaba Tongyi Lab)

`github.com/datajuicer/data-juicer`

**Enterprise Stack Role: Multi-Format Ingestion Pipeline & Raw Data Streamliner.**

Provides 200+ sandboxed operators to format, deduplicate, and structurally slice chaotic streams (JSON, logs, PDFs, telemetry) into stable LLM-readable inputs without enforcing rigid destination schemas. Use versioned domain packs of operators—not a second warehouse ETL engine.

*Puzzle piece: raw byte standardization; lightweight alternative to heavyweight Spark/dbt for semantic prep.*

### 4. PageIndex (by VectifyAI)

`github.com/VectifyAI/PageIndex`

**Enterprise Stack Role: Reasoning-Based Document & Structural Tree Navigator.**

Bypasses high-cost dense vector monocultures by building layout-aware coordinate index trees on documents, enabling agents to navigate to structural text positions dynamically with lightweight routing models.

*Puzzle piece: cost and scale limits of dense vectors; structure-first retrieval for long documents.*

## 4. Reference Architecture: Full Stack Map

Moving away from structured ETL requires stitching specialized building blocks into a unified Semantic Data Core:


## 5. Enterprise Subsystems Beyond the Four Engines

Org-wide operation requires additional planes the original engine list does not cover:

| Subsystem | Why Required |
|---|---|
| **Connector & CDC fabric** | Org data is continuous, not one-shot dumps; watermarks and late data are normal. |
| **Entity resolution (ER)** | Same customer/product/employee across 50–500 systems; without ER, graphs become duplicate islands. |
| **Conflict & discrepancy engine** | Explicit multi-source truth; never silent winners on regulated predicates. |
| **Ontology registry** | Layered L0 universal / L1 domain / L2 team extensions; promotion path to governed types. |
| **Query router + budget governor** | Prevents “run all models every time” cost blowups; enforces IDF and latency classes. |
| **Policy / ABAC engine** | Block-level security without classical tables; intersection propagation to derivatives. |
| **Eval & regression harness** | Only way schema-on-read is trustworthy; golden sets and promotion gates. |
| **Human adjudication UI/API** | Pin, merge, reject facts; audited higher-trust edges. |
| **Observability & FinOps** | Token $, latency, cache hit rate, extraction errors, per-tenant kill switches. |
| **Semantic view materializer** | Optional: emit stable warehouse views when a semantic contract hardens (BI escape hatch). |

## 6. The Control Plane: Mathematical Guardrails & Specifications

### 6.1 Mathematical Token Budgeting & Routing (IDF)

To prevent large context windows from causing cost escalations or execution timeouts, the query routing subsystem evaluates an explicit **Information Density Factor (IDF)** over candidate chunks. The routing optimization engine minimizes costs ($V_c$) and latency ($V_l$) while ensuring the confidence metric ($C_f$) tracks above an enterprise target threshold ($T$):

$$\textbf{Minimize: } \alpha V_c + \beta V_l \textbf{ subject to } C_f \geq T$$

The Information Density Factor evaluates how structurally dense extracted information is relative to its raw token payload footprint:

**IDF = (Count of Evaluated Predicates) / (Total Token Count)**

"Evaluated predicates" = entity-attribute claims already extracted into the graph for the candidate span.

| IDF Band | Routing Behavior |
|---|---|
| **High density (IDF ≥ 0.75)** | Routed away from long-context LLMs; execute via localized cross-encoder reranking and graph entity cards. |
| **Mid density (0.25 ≤ IDF < 0.75)** | Hybrid: graph facts + PageIndex leaf spans + selective mid-tier LLM synthesis under budget. |
| **Low density (IDF < 0.25)** | PageIndex structural layout maps isolate token depth to specific leaf indices before any large-model call. |
| **Global / thematic intent** | GraphRAG community summaries (prefer precomputed/async); sample evidence under budget. |

### 6.2 Query Router Decision Table

| Intent Signal | Primary Path | Secondary |
|---|---|---|
| **"What is X?" entity lookup** | Graph entity card | PageIndex for source docs |
| **"What changed for X?"** | Temporal graph | Raw episodes in time window |
| **"Top themes / failure modes"** | GraphRAG communities (async if cold) | Sampled evidence |
| **"Find in this 400-page contract"** | PageIndex | Sparse keyword |
| **Hybrid / ambiguous** | Parallel fan-out under budget | Rerank + fuse + conflict surfacing |

**Latency classes:** Interactive (seconds) · Standard (tens of seconds) · Deep (async job + webhook). Budget exhausted → partial answer + continue-deep option.

### 6.3 Operational Conflict Resolution Matrix

Project Synapse does **not** filter out conflicting data records at write time. Instead, it computes a dynamic **Validity Weight ($W_v$)** at the query layer using Authority Rank ($A_r$), Lineage Proximity ($L_p$), and a Temporal Decay coefficient ($\lambda$):

$$W_v = (A_r \times e^{-\lambda \cdot \Delta t}) + L_p$$

| Conflict Typology | Technical Core Challenge | System Behavior |
|---|---|---|
| **Scalar Clash** | Opposing numeric metrics reported by distinct operational nodes in the same time frame. | Flags as AMBIGUOUS_CONFLICT; returns both values with source lineages unless human_pin is set. |
| **Temporal Supersession** | Downstream CDC events reporting conflicting states over an identical entity instance. | If the newer event scores higher W_v, current view shifts while past state remains a historic edge. |
| **Domain Overlap** | Opposing attribute states across cross-functional barriers (e.g., CRM vs Billing). | Prefer via Ontology Registry authority maps; for regulated predicates still surface conflict—do not silently hide alternatives. |
| **Compatible Multi-Value** | Multiple valid emails, regions, or product names. | Keep set; status may be accepted_plural. |
| **Human Pin** | Expert adjudication required for high-impact entities. | Status resolved with adjudicator, reason, audit trail; elevated trust weight. |

**Query default:** best-effort synthesis with uncertainty—never fake consensus.

### 6.4 ABAC Propagation (Security Follows the Block)

Unstructured storage blocks carry an immutable Attribute-Based Access Control (ABAC) manifest. When summaries or facts are derived by reasoning nodes, the engine applies a mathematical intersection of source rights:

$$\text{Derived Fact Key} = ACL_{\text{Object 1}} \cap ACL_{\text{Object 2}} \cap \ldots \cap ACL_{\text{Object n}}$$

If a querying user's authorization manifest fails to match the evaluated derived fact key, the node **drops from the model attention window before prompt compilation**—not merely redacted after generation. Attributes typically include: domain,

team, region, sensitivity, legal_hold. Optional envelope encryption per sensitivity class; no cross-ACL aggregation unless policy explicitly allows (anti-inference).

## 7. Core Schema Specifications

Canonical objects for the semantic data plane. Machine-readable contracts; expand via ADR as domains onboard. Full set: RawObject, Episode, Entity, Fact, Conflict, Claim.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "SynapseDataCore",
  "defs": {
    "RawObject": {
      "required": ["object_id","source_system","content_hash",
                   "ingested_at","bytes_ref","acl_tags"],
      "properties": {
        "object_id": "uuid", "source_system": "string",
        "source_uri": "string", "content_hash": "string",
        "ingested_at": "date-time", "bytes_ref": "uri",
        "media_type": "string", "acl_tags": ["string"],
        "sensitivity": "string", "retention_class": "string"
      }
    },
    "Episode": {
      "required": ["episode_id","raw_object_ids","domain",
                   "prep_pipeline_version","acl_tags"],
      "properties": {
        "episode_id": "uuid", "raw_object_ids": ["uuid"],
        "domain": "string", "time_span": "object",
        "prep_pipeline_version": "string",
        "payload_ref": "uri", "quality_signals": "object",
        "acl_tags": ["string"]
      }
    },
    "Entity": {
      "required": ["entity_id","entity_type","status"],
      "properties": {
        "entity_id": "uuid", "entity_type": "string",
        "canonical_name": "string", "aliases": ["string"],
        "external_ids": ["{system,id}"],
        "trust_score": "0..1",
        "status": "active|merged|deprecated"
      }
    },
    "Fact": {
      "required": ["fact_id","subject_entity_id","predicate",
                   "object","confidence","evidence_refs"],
      "properties": {
        "fact_id": "uuid", "subject_entity_id": "uuid",
        "predicate": "string", "object": "any",
        "valid_from": "date-time", "valid_to": "date-time|null",
        "confidence": "0..1", "extractor_version": "string",
        "evidence_refs": ["uuid"]
      }
    },
    "Conflict": {
      "required": ["conflict_id","subject_entity_id","predicate",
                   "competing_fact_ids","status"],
      "properties": {
        "conflict_id": "uuid", "subject_entity_id": "uuid",
        "predicate": "string",
        "competing_fact_ids": ["uuid"],
        "status": "open|resolved|accepted_plural",
        "resolution": "object"
      }
    },
    "Claim": {
      "required": ["statement","supporting_fact_ids","confidence"],
      "properties": {
        "statement": "string",
        "supporting_fact_ids": ["string"],
        "raw_citations": ["string"],
        "confidence": "0..1",
        "uncertainty_notes": ["string"],
        "policy_filtered": "boolean"
      }
    }
  }
}
```

### 7.1 Entity Resolution at Scale

1. **Blocking** — cheap keys (email domain, tax-id hash, normalized name n-grams)
2. **Pairwise scoring** — local model / rules
3. **Clustering** — merge candidates within domain + cross-domain links
4. **Human review queues** — high-impact entity types only

5. **Stable IDs** — merges create redirect edges; never delete history

### 7.2 Connector Taxonomy (Variety)

| Class | Examples | Ingest Mode | Prep Needs |
|---|---|---|---|
| **Structured OLTP** | Postgres, SaaS CRM | CDC / snapshot | Light typing, PII tag |
| **Semi-structured** | JSON events, webhooks | Stream | Schema drift detect |
| **Logs / traces** | App, security | Stream + sample | Slice, redaction |
| **Documents** | PDF, slides, contracts | Batch | PageIndex trees |
| **Communications** | Tickets, email meta | Sync | Thread reconstruction |
| **Metrics** | TSDB, billable usage | Rollup + ref | Align to entities |
| **Tribal / messy** | Spreadsheets, notes | Drop zone | High discrepancy rate |

## 8. Structural Gaps Closed: Production Hole Register

Original blueprint gaps plus org-wide holes. Each has a designed mitigation owner.

| ID | Hole | Mitigation Design |
|---|---|---|
| **H1** | Deterministic precision | Dual-path extract (parsers + LLM residual); constrained JSON; verifier; numeric claims need evidence; regulated predicates need authority or human pin. |
| **H2** | Inference-time latency | Latency classes; semantic key cache; entity cards; speculative precompute; early-exit when $C_f \geq T$. |
| **H3** | Token economics | IDF routing; tiered models; hard per-tenant budgets; claim cache with TTL; domain partitioning. |
| **H4** | Access control | Tag-at-ingest; ACL intersection on derivatives; pre-prompt filter; audit log; optional envelope encryption. |
| **H5** | Schema drift | Catalog watches; reprocess jobs; predicate versioning—not silent overwrite. |
| **H6** | Idempotency | content_hash + pipeline_version define episode identity; temporal supersession of facts. |
| **H7** | Freshness / CDC | Watermarks; as_of queries; late data creates temporal corrections with lineage. |
| **H8** | Multi-domain ontology | L0 universal / L1 domain packs / L2 team extensions; registry promotion. |
| **H9** | Retrieval path conflict | Query router (§6.2); never run all engines unbounded on every query. |
| **H10** | Eval & regression | Golden sets per domain; citation precision; conflict honesty rate; CI gates. |
| **H11** | Human-in-the-loop | Adjudication queues for ER merges, conflicts, ontology promotion; audited pins. |
| **H12** | Observability & FinOps | RED metrics; $ per 1k queries; cache hit ratios; tenant kill switches. |
| **H13** | DR & retention | Raw is backup root; graph rebuildable from raw + pipeline versions; legal hold & erasure via lineage. |
| **H14** | Model supply chain | Pinned model versions; residency constraints; redaction pre-egress. |
| **H15** | Write-back / activation | Read-mostly; optional action bus with human approval for high-risk SoR mutations. |
| **H16** | BI coexistence | Materialize stable high-trust semantic views into warehouse tables as products of the graph. |

## 9. Query Lifecycle (End-to-End)

1. Authenticate + load user/tenant policy context
2. Parse intent → query class + budget class
3. Policy-scoped retrieval plan (router + IDF)
4. Parallel fetch under budget: entity/fact graph · doc trees · community summaries · optional vector/keyword
5. Conflict-aware fusion (never hide open conflicts for regulated predicates)
6. Structured answer + citations + confidence + gaps (Claim object)

7. Cache claim (TTL, ACL-bound)
8. Emit telemetry + optional human feedback signal

> **Designed Failure Modes**
> Budget exhausted → partial answer + "continue deep job". Policy blocks evidence → refuse or redacted synthesis. Low confidence → escalate / clarify. Open conflict → present both sides with authority ranking.

## 10. Prototype Testing Path: Infrastructure & Operational Incidents

The ideal validation landscape for Project Synapse is **Infrastructure & Operational Incidents**. This domain naturally produces chaotic, conflicting data profiles where systems disagree continuously during an incident event—for example:

- Automated deployment servers reporting clean build metrics
- Kubernetes clusters outputting crash loops and restarts
- Engineering groups flagging rollbacks manually in chat rooms
- Monitoring tools emitting partial or delayed alerts

By building the initial playground within this vertical, we demonstrate how schema-on-read systems reconstruct complex incidents without expensive data synchronization layers—while exercising ER, conflict flags, PageIndex on runbooks, and temporal graph queries ("What changed for service X in the last 30 minutes?").

### Paper-Only Proofs Available Before Infra

- Contract completeness review (every hole has an owner subsystem)
- Threat model (ACL leak, hallucination, cost runaway, graph poisoning)
- Query router decision tables with cost/latency envelopes
- Synthetic discrepancy corpora (deploy-green vs crash-loop vs chat-rollback)
- Golden-set design and promotion gates
- Reference cost model (tokens/query class × expected QPS)

## 11. Doability Assessment

| Capability | Feasibility | Residual Risk |
|---|---|---|
| **Multi-format landing + prep** | High | Operator sprawl |
| **Continuous temporal graph** | Medium–High | Extraction cost at stream volume |
| **Entity resolution org-wide** | Medium | Long-tail identity mess |
| **Conflict-aware truth** | Medium | Product/UX complexity |
| **Structure-aware doc retrieval** | High | Less mature than vectors |
| **Global theme synthesis** | Medium | Expensive; stale summaries |
| **ABAC on unstructured** | Medium | Inference side-channels |
| **Deterministic numeric claims** | Medium | Never 100% without system of record |
| **Interactive latency at scale** | Medium | Deep queries still async |
| **Token cost control** | High if budgets enforced | Culture of unbounded large-model calls |
| **Full org single brain day-1** | Low | Must domain-slice |

> **Verdict**
> **Doable** as a multi-year platform with phased domain rollout, coexisting with warehouses. **Not doable** as a single big-bang "replace the warehouse next quarter" project. Most doable path: semantic intelligence plane alongside existing lake/warehouse, absorbing messy, cross-domain, and agentic use cases that classical ETL handles poorly.

## 12. Implementation Roadmap

### Demo Path (near-term, low infra)

• **Phase 0:** Architecture ADR sign-off, schema locking (this document), validation mockups, golden-set design for infra/ops incidents.

• **Phase 1:** Data-Juicer (or equivalent) ingestion scripts landing raw streams into an object store with ACL tags and content hashes.

• **Phase 2:** Graphiti instance mapping dynamic entities, temporal edges, and conflict flags on read; basic ER for service/deploy IDs.

• **Phase 3:** Front-end query UI (e.g., Vercel) utilizing long-context engines (e.g., Google AI Studio / approved LLM endpoints) with citation display and budget meters.

### Platform Path (org-wide maturity)

• **Phase 4:** Multi-source discrepancy + human adjudication queues; ABAC enforcement in router; eval harness in CI; PageIndex on runbooks/PDFs.

• **Phase 5:** Multi-domain ontology layers; GraphRAG offline community jobs; FinOps dashboards and tenant kill switches.

• **Phase 6:** Semantic view materializer (BI escape hatch); connector catalog; self-serve domain onboarding; DR rebuild drills from raw.

• **Phase 7:** Org-default path for selected question classes; warehouse coexistence policy codified; regulated-predicate governance live.

| Phase | Exit Criteria |
|---|---|
| **P0** | All H1–H16 owned; schemas locked; first vertical scenario walkthrough complete on paper |
| **P1–P3** | End-to-end demo query over incident corpus with citations and at least one open conflict surfaced |
| **P4–P5** | Eval gates green on golden set; ABAC negative tests pass; cost per query class within envelope |
| **P6–P7** | Second domain onboarded without schema rewrite; rebuild-from-raw drill succeeds |

## 13. Why This Is the Future of Data Management

| Era | Paradigm | Weakness |
|---|---|---|
| **Warehouse** | Schema-on-write | Brittle, slow to new questions, context loss |
| **Lakehouse** | Cheap storage + SQL | Still schema-heavy for meaning |
| **Classical RAG** | Chunk + embed | Weak global, temporal, and conflict handling |
| **Synapse** | Raw + temporal graph + structure + conflict + policy | Harder platform; better fit for agentic org cognition |

> **Future-State Claim (Precise)**
> Organizational data management shifts from “clean once into tables” to “land once with policy and lineage, continuously extract meaning, resolve identity and conflict explicitly, and answer under budget with citations.” That is a platform shift—not a tool swap.

## 14. Open Decisions & Immediate Workstreams

### Open Decisions

1. Primary first domain deep-dive details (infra/ops incidents confirmed as default)
2. Graph store assumption (managed vs self-hosted; partition strategy)

3. How much vector search is allowed alongside PageIndex (hybrid default recommended)
4. Authority ranking owners per predicate (data governance RACI)
5. Model residency constraints for target org class
6. Coexistence policy with existing warehouse / lakehouse (parallel forever vs gradual absorb)

### Paper-Only Workstreams (No Infra Required)

| # | Workstream | Deliverable |
|---|---|---|
| **A** | Architecture ADRs | One ADR per hole H1–H16 with chosen pattern |
| **B** | Canonical contracts | Frozen JSON schemas for all core objects |
| **C** | Query router spec | Intent taxonomy + IDF bands + budgets |
| **D** | Discrepancy playbook | Conflict types + resolution policies by domain |
| **E** | Security model | ABAC attributes, propagation, anti-inference rules |
| **F** | Eval blueprint | Metrics, golden set design, promotion gates |
| **G** | Cost & latency envelopes | Spreadsheet model by query class |
| **H** | Threat model | STRIDE-style on the semantic plane |
| **I** | Incident walkthrough | Full paper scenario: green deploy vs crash loop vs chat rollback |

## 15. Bottom Line

• **Org-wide, messy, discrepant data** is the correct target—and it forces first-class entity resolution, conflict storage, ABAC, budgets, and eval—not merely “LLM over a lake.”

• The four open-source engines (Graphiti, GraphRAG, Data-Juicer, PageIndex) are **necessary but not sufficient**; the control plane and discrepancy model make the system enterprise-real.

• **Doable:** yes, as a phased platform coexisting with warehouses, if H1–H16 are treated as core product.

• **Not doable:** as magic zero-work, zero-cost, zero-governance replacement of all ETL.

• **Work now:** plug every remaining open decision in design, contracts, and proof methods—so when infra appears, build is assembly, not invention.

---

Project Synapse — Unified Master Architectural Specification
Consolidated from Zero_ETL_Semantic_Data_Blueprint.pdf, Project_Synapse_Master_Architecture.pdf (Gemini), and ORG_WIDE_SEMANTIC_DATA_CORE.md. Theme aligned to Gemini master visual language. Document class: architecture thesis / production-completeness specification.